Computación de Alto Rendimiento (HPC)

Caso de Estudio 2 — Concurrencia con OpenMP,
Perfilado y Benchmark

Paralelización con OpenMP de la multiplicación de matrices,

Caracterización de CPU y memoria, y
comparación de rendimiento
entre dos máquinas

José Alejandro Quintero Gahona

Meycol Kleym Quintero Castaño

María Angélica Alvarez Giraldo

Docente: Ramiro Andrés Barrios Valencia

Asignatura: Computación de Alto Rendimiento

28 de octubre de 2026

Índice

1. Introducción **3**

2. Objetivos **3**
2.1. Objetivo general 3
2.2. Objetivos específicos 4

3. Marco teórico **4**
3.1. OpenMP: qué es y qué aporta 4
3.2. Los tres niveles de un programa paralelo 5
3.3. Métricas de rendimiento: MIPS, FLOPS y por qué no sirven aquí 5
3.4. Benchmark: definición y tipos 6
3.5. Perfilado: definición y tipos 6
3.6. La jerarquía de caché y el miss rate 7

4. El programa base (versión secuencial) **7**
4.1. Qué hace el programa 7
4.2. El algoritmo en código 8
4.3. El patrón de acceso a memoria 8
4.4. Estructura modular del código 9

5. Medición del tiempo **9**
5.1. Wall clock vs tiempo de CPU 9
5.2. Qué opciones hay en C 10
5.3. Por qué `CLOCK_MONOTONIC_RAW` 10
5.4. Qué se cronometra y qué no 10
5.5. Por qué se repite el experimento 10 veces 10
5.6. El orden de los bucles importa 11

6. Implementación con OpenMP **11**
6.1. Del diseño manual al declarativo 11
6.2. La directiva elegida 11
6.3. Justificación a partir de los ejemplos de la especificación 12
6.4. Por qué cada elemento de la directiva 12
6.5. Por qué no se necesitan otras cláusulas 12
6.6. Configuración del número de hilos 13
6.7. El flag de compilación no es opcional 13

7. Benchmark: protocolo y marco de comparación **14**
7.1. Qué se mide y por qué el kernel 14
7.2. Qué se compara entre las dos máquinas 14
7.3. El protocolo para garantizar comparabilidad 14
7.4. Qué se captura del hardware 15

8. Perfilado de la aplicación **15**
8.1. Objetivo del perfilado . . . 15
8.2. Perfilado de CPU con GProf . . . 16
8.3. Perfilado de memoria con Cachegrind . . . 16
8.4. Footprint de memoria con `getrusage` . . . 17
8.5. Tamaño de N usado para el perfilado . . . 17

9. Diseño experimental **17**
9.1. Variables . . . 17
9.2. Metodología: 10 corridas por configuración . . . 17
9.3. Orden de los bucles . . . 18
9.4. Correctitud: validación contra una implementación de referencia . . . 18
9.5. Reproducibilidad del benchmark entre máquinas . . . 18
9.6. Herramientas . . . 19

10. Resultados **19**
10.1. Perfilado de CPU: dónde se va el tiempo . . . 19
10.2. Perfilado de memoria: el comportamiento de la caché . . . 19
10.3. Footprint de memoria . . . 20
10.4. Desempeño: las cinco estrategias comparadas . . . 21
10.5. Escalamiento de OpenMP . . . 21
10.6. Comparación entre las dos máquinas . . . 22

11. Análisis de resultados **23**
11.1. El perfilado confirma la hipótesis del caso de estudio 1 . . . 23
11.2. Por qué OpenMP no supera al kernel transpuesto . . . 24
11.3. Por qué el speedup crece con el tamaño de N . . . 24
11.4. El punto donde T=16 deja de ayudar . . . 25
11.5. Comparación con la paralelización manual . . . 25
11.6. El error metodológico que casi distorsionó el resultado . . . 26
11.7. Qué aporta OpenMP frente a la paralelización manual . . . 26

12. Conclusiones **26**
12.1. Logros del trabajo . . . 26
12.2. Hallazgos principales . . . 27
12.3. Limitaciones . . . 27
12.4. Trabajo futuro . . . 28

13. Referencias **28**

1. Introducción

Este documento presenta el trabajo realizado sobre el caso de estudio 2 de la materia de Computación de Alto Rendimiento (HPC). La consigna de la actividad pide tres cosas: implementar **concurrencia mediante OpenMP** sobre la aplicación de multiplicación de matrices, realizar la **caracterización (perfilado) mínimo de la aplicación** en CPU y memoria, y **medir el rendimiento en dos máquinas diferentes** (benchmark), mostrando los resultados de ambas máquinas en el documento.

El trabajo parte del programa secuencial de multiplicación de matrices ya estudiado en el caso de estudio 1, donde se implementaron versiones paralelas con **POSIX Threads** y con **procesos** `fork()` usando memoria compartida POSIX. Ese caso de estudio quedó además acompañado con una comparación de *optimizaciones*: flags de compilación agresivos (`-O3 -march=native -floop-interchange -funroll-loops -floop-nest-optimize`) y un kernel transpuesto que cambia el patrón de acceso a la matriz B para aprovechar mejor la caché.

En este caso de estudio 2 se cierra el círculo. La pregunta central que responde el trabajo es: **¿qué aporta una API de paralelización de alto nivel como OpenMP frente a la paralelización manual con hilos?**, y para responderla con evidencia y no solo con intuición se añaden dos instrumentos que el caso de estudio 1 no tenía:

- El **perfilado**, que permite observar *dónde* se va el tiempo y *por qué* una versión es más rápida que otra. En particular, permite medir la tasa de fallos de caché (*cache miss rate*) de cada versión del kernel, que es la variable que explica el comportamiento observado en el caso de estudio 1.
- El **benchmark entre máquinas**, que permite poner el mismo código en hardware diferente y observar cuánto de la ganancia depende del algoritmo y cuánto depende del equipo.

El resto del documento está organizado así: la sección 2 enumera los objetivos; la sección 3 presenta los conceptos teóricos mínimos (OpenMP, benchmark, perfilado, métricas de rendimiento); la sección 4 describe el programa base secuencial; la sección 5 explica la metodología de medición de tiempo; la sección 6 describe la implementación con OpenMP y la justificación de las directivas elegidas; la sección 7 introduce el marco teórico del benchmark y del perfilado; la sección 8 muestra los resultados del perfilado; la sección 9 explica el diseño experimental; la sección 10 presenta los resultados de desempeño en ambas máquinas; la sección 11 los analiza; y la sección 12 cierra con las conclusiones.

2. Objetivos

2.1. Objetivo general

Completar el caso de estudio 2 implementando concurrencia mediante **OpenMP** sobre la aplicación de multiplicación de matrices, caracterizar el comportamiento de la aplicación a nivel de CPU y de memoria, y medir su rendimiento en dos máquinas diferentes para poder comparar hardware.

2.2. Objetivos específicos

- Implementar la versión concurrente con **OpenMP** partiendo del programa secuencial, aplicando las directivas de la API (`#pragma omp`) de forma justificada.
- Justificar técnicamente la **elección de cada directiva y cláusula** utilizada, apoyándose en los ejemplos de la especificación oficial de OpenMP.
- Realizar el **perfilado mínimo de la aplicación** en dos ejes: **CPU** (tiempo por función, con GProf) y **memoria** (tasa de fallos de caché y footprint de memoria, con Cachegrind y `getrusage`).
- **Medir el rendimiento en dos máquinas distintas** con Linux, usando un benchmark reproducible, y comparar los resultados mediante la métrica de diferencia porcentual de rendimiento.
- Analizar de forma integrada los resultados de *benchmarks*, *profiling* y *speedup* para explicar qué limita el rendimiento de la aplicación.

3. Marco teórico

En esta sección se introducen los conceptos mínimos necesarios para entender el resto del documento. El objetivo es tener un vocabulario común, no hacer desarrollos matemáticos largos.

3.1. OpenMP: qué es y qué aporta

OpenMP es una **API (Application Programming Interface) diseñada para la computación paralela en arquitecturas de memoria compartida**. No es un lenguaje ni una biblioteca: es un conjunto de **directivas de compilador**, **rutinas de biblioteca** y **variables de ambiente** que permiten al programador describir paralelismo sin escribir explícitamente la creación y sincronización de hilos.

El elemento central es la **región paralela**. Cuando el hilo maestro encuentra una región paralela, crea un *equipo de hilos* (*team*), todos ejecutan el bloque de código de la región, y al final de la región el hilo maestro espera a que todos terminen. Esa sincronización final es lo que garantiza que los resultados estén listos cuando la ejecución continúa.

La analogía con la versión de pthreads del caso de estudio 1 es directa: allí se escribía a mano el `pthread_create` para cada hilo y el `pthread_join` para esperarlos. OpenMP hace exactamente eso internamente, pero lo hace el *runtime*.

Tres conceptos gobiernan el comportamiento de una región paralela:

- **Directivas y centinelas**. Una directiva es una línea de código con significado *solo* para el compilador. Se escriben con el centinela `#pragma omp`. Un punto fundamental: **si el programa se compila sin la bandera `-fopenmp`, las directivas se ignoran silenciosamente** y el código se ejecuta secuencialmente. Por eso el flag de compilación es parte inseparable del uso de OpenMP.

- **Datos compartidos y privados**. En una región paralela, por defecto las variables son *compartidas* (todos los hilos ven la misma copia) salvo las variables de control de los bucles, que son *privadas* por defecto. El programador puede declarar explícitamente qué es compartido y qué es privado con las cláusulas `shared()` y `private()`.
- **Sincronización**. La región paralela tiene una *barrier* implícita al final. Además existen construcciones explícitas (`critical`, `atomic`, `barrier`) para cuando varios hilos deben tocar el mismo dato.

3.2. Los tres niveles de un programa paralelo

En el caso de estudio 1 se implementó la paralelización manual, es decir, el nivel más bajo: el programador decide exactamente qué hace cada hilo. OpenMP introduce un nivel intermedio en el que el programador *describe* qué bucles paralelizar y el *runtime* decide el reparto. Existen además niveles más altos (*tasking*, *worksharing*) que no se usaron en este trabajo.

3.3. Métricas de rendimiento: MIPS, FLOPS y por qué no sirven aquí

El **rendimiento computacional** se puede medir de dos formas distintas:

- **Tiempo de respuesta**: cuánto tarda el sistema en resolver *una* aplicación concreta. Es la métrica del usuario final. Es la que se usa en este trabajo.
- **Throughput (productividad)**: cuánto trabajo es capaz de completar el sistema en un intervalo de tiempo. Es una métrica del *sistema*, más cercana a la de un servidor con many workloads.

Para comparar dos máquinas se define el rendimiento como $R = 1/T$ y la diferencia porcentual:

$$\%\Delta = \frac{R_x - R_y}{R_y} \times 100$$

Las unidades de medida históricas para cuantificar el rendimiento computacional son:

- **MIPS** (millions of instructions per second):Instructions per second. Es la unidad clásica, pero **depende fuertemente del repertorio de instrucciones** de cada microarquitectura, por lo que es poco comparable entre arquitecturas distintas.
- **FLOPS** (floating point operations per second): operaciones de coma flotante por segundo. Es la unidad moderna y se usa en el ranking *Top500*. El problema es que **solo aplica a operaciones de coma flotante**.

En este trabajo el kernel trabaja con **enteros**, por lo que **no se puede reportar la aplicación en FLOPS**. Esta es una limitación real y no un descuido: la alternativa sería reescribir el kernel con `double`, lo que cambiaría el objeto de estudio (el algoritmo de punto flotante tiene sus propios problemas de precisión y de uso de la FPU) sin aportar nada a la pregunta que se quiere responder. Por eso se reportan **tiempos, speedup y diferencia porcentual de rendimiento**, que además es lo que el enunciado de la actividad pide de forma explícita.

3.4. Benchmark: definición y tipos

Un **benchmark** es un programa que permite evaluar el rendimiento de una arquitectura o de un equipo. Su lógica es generar una *carga de trabajo* conocida y estandarizada para la cual se puede predecir el rendimiento en un trabajo real.

Según la procedencia de la carga de trabajo, los benchmarks se clasifican en:

- **Programas reales completos**: por ejemplo un compilador, un procesador de textos, un outil CAD. Son los más representativos.
- **Kernels extraídos de programas reales**: es la categoría de este trabajo. Se extrae la parte central (el *kernel*) de un programa real que es la que consume la mayor parte del tiempo de ejecución. El profesor insistió en medir el tiempo del *kernel* de la multiplicación de matrices, no el programa completo, porque es el kernel lo que domina el tiempo y lo que determina el comportamiento de la caché.
- **Programas reducidos o *toys***: programas muy pequeños, de 10 a 100 líneas, que generan carga sin resolver un problema real.
- **Programas sintéticos**: creados artificialmente para generar carga en un recurso específico.

Una advertencia importante del profesor: los benchmarks reducidos y los sintéticos **no cargan la memoria principal de forma realista**, por lo que no son una buena opción para caracterizar el desempeño de una aplicación real. Para eso hay que usar programas reales o *kernels* extraídos de problemas reales, como el nuestro.

3.5. Perfilado: definición y tipos

El **perfilado** (*profiling*) tiene un objetivo distinto al benchmark. El benchmark mide el *desempeño*; el perfilado busca **identificar el comportamiento del programa en una arquitectura específica**: dónde están los puntos ineficientes, dónde hay embotellamiento de memoria o de acceso a disco, y qué explica el escalamiento de la solución.

Las herramientas de perfilado se clasifican por su mecanismo:

- **Contadores de hardware**: leen directamente los contadores del procesador (ciclos, instrucciones, fallos de caché).

- **Muestreo (*sampling*)**: interrumpen el programa en intervalos periódicos y capturan el estado, para después hacer un análisis estadístico. Es el mecanismo de `perf` y de `gprof`.
- **Trazado (*tracing*)**: en lugar de interrumpir periódicamente, detectan la ocurrencia de un evento concreto (por ejemplo, una llamada a función) y lo registran.

En este trabajo se usan dos herramientas, que cubren los dos ejes pedidos por la consigna:

- **GProf**: perfilador por muestreo que se integra con el compilador. Se activa con la bandera `-pg`, que inserta instrumentación en el programa. Al ejecutarse, el programa escribe un archivo `gmon.out` con los contadores recolectados; luego la utilidad `gprof` convierte ese archivo binario en un reporte legible. Es la herramienta que el profesor presentó en clase.
- **Cachegrind**: perfilador de simulación de caché, parte de Valgrind. Reproduce el comportamiento de una jerarquía de caché (L1, LL) durante la ejecución y reporta, para cada función, cuántas instrucciones se ejecutaron y cuántas fallaron en caché. Con la opción `-cache-sim=yes` reporta además las tasas de fallo (*miss rate*) de lectura y escritura.

3.6. La jerarquía de caché y el miss rate

El procesador no accede directamente a la memoria RAM: hay una jerarquía de cachés, pequeños y muy rápidos, entre el núcleo y la RAM.

Nivel	Tamaño típico	Tiempo de acceso
L1 (datos e instrucciones)	32–64 KiB por núcleo	4 ciclos
L2	512 KiB–2 MiB por núcleo	12 ciclos
L3 (compartida)	8–32 MiB	40 ciclos
RAM	GiB	200 ciclos

El **miss rate** es la fracción de accesos a un nivel de caché que no se encuentran en ese nivel y deben bajar al nivel inferior. Es la métrica clave para entender el rendimiento de un kernel: un miss a RAM cuesta cientos de ciclos, mientras que un acierto a L1 cuesta unos pocos.

Cuando se dice que un acceso es *contiguo* o *secuencial*, se está describiendo el patrón con que se recorre un arreglo en memoria. En el algoritmo clásico de multiplicación de matrices, el acceso a B no es contiguo, y esa es la raíz del problema que se mide en la sección 8.

4. El programa base (versión secuencial)

4.1. Qué hace el programa

El programa toma dos matrices cuadradas A y B de tamaño $N \times N$ y calcula la matriz producto $C = A \times B$ con el algoritmo clásico. Cada elemento se calcula como:

$$C[i][j] = \sum_{k=0}^{N-1} A[i][k] \cdot B[k][j]$$

Hay N^2 elementos a producir y cada uno requiere N multiplicaciones y $N-1$ sumas, así que la complejidad es $O(N^3)$. Para $N = 1000$ son 10^9 operaciones; para $N = 4000$, $6{,}4 \times 10^{10}$.

4.2. El algoritmo en código

El bucle que después se paraleliza en todas las versiones es un triple bucle anidado:

```
for (int i = 0; i < N; i++)
    for (int j = 0; j < N; j++)
        for (int k = 0; k < N; k++)
            C[i*N + j] +=
                A[i*N + k] *
                B[k*N + j];
```

Los índices `i*N + k` y `k*N + j` convierten el índice bidimensional `[i][k]` en uno unidimensional `[i*N+k]`, porque las matrices se almacenan en un bloque contiguo de memoria (allocado con `malloc`), no como arreglos bidimensionales de C.

4.3. El patrón de acceso a memoria

Este es el detalle que resultará central en el análisis, así que conviene hacerlo explícito desde el inicio. En el bucle interior, con i y j fijos, el índice `k` recorre:

- $A[i \cdot N + k]$: acceso **contiguo**. Al avanzar k, la dirección avanza de a un entero. La fila i de A se recorre secuencialmente.
- $B[k \cdot N + j]$: acceso **con salto**. Al avanzar k, la dirección avanza N enteros. La columna j de B se recorre *a saltos*, y cada salto de N enteros cae normalmente en una línea de caché diferente.
- $C[i \cdot N + j]$: acceso **constante**. Todos los k escriben en la misma celda, que queda en un registro.

Es decir: la mitad del trabajo de la multiplicación (la lectura de B) accede a la memoria en un orden que no corresponde al orden en que los datos están almacenados. Este patrón es el que se mide en la sección 8 con Cachegrind y el que explica, en la sección 11, por qué la transposición de B produce una ganancia tan grande sin necesidad de paralelizar.

4.4. Estructura modular del código

El proyecto está organizado en módulos, de manera que el kernel de multiplicación (que es lo que cambia entre versiones) está aislado del resto:

- `modules/memoria.c`: reserva memoria para las tres matrices con `malloc` y verifica que la reserva haya sido exitosa.
- `modules/llenado.c`: llena A y B con valores aleatorios usando `srand`/`rand` y pone C en cero.
- `modules/tiempo_mult.c`: contiene la función `multiplicar_y_medir` que ejecuta el triple bucle secuencial y mide el tiempo con `clock_gettime`.
- `modules/tiempo_mult_trans.c`: variante con B transpuesta, que accede a B por fila.
- `modules/tiempo_mult_openmp.c`: variante con la directiva `#pragma omp parallel for collapse(2)`, que es la implementación de este caso de estudio.
- `src/matriz_monotonic_openmp.c`: programa principal que usa el kernel con OpenMP.

El *headers* correspondientes viven en `include/`. Esta separación permite medir el *kernel* de forma aislada, que es lo que interesa para el perfilado.

5. Medición del tiempo

5.1. Wall clock vs tiempo de CPU

En cualquier sistema operativo hay dos formas de medir cuánto tarda un programa:

- **Wall clock**: el tiempo total transcurrido desde que el programa arranca hasta que termina. Es lo que un usuario percibe como "cuánto tardó". Incluye las esperas por E/S, las pausas por quantum y cualquier otra cosa que pase en el sistema.
- **Tiempo de CPU**: el tiempo que la CPU pasó efectivamente ejecutando nuestro programa. Se subdivide en tiempo de **usuario** (código nuestro) y tiempo de **sistema** (atendiendo llamadas al kernel).

Durante la materia, el profesor indicó que la métrica correcta para comparar rendimiento entre programas es el **wall clock**, porque refleja la experiencia real del usuario. El tiempo de CPU mide otra cosa (cuánto trabajó el procesador), pero no cuánto tardó el programa en completarse. En este trabajo se mide con wall clock en todos los casos.

5.2. Qué opciones hay en C

El lenguaje C ofrece varias funciones de tiempo. Esta tabla resume las que se consideraron:

Función	Qué mide	Wall clock?
`clock()`	Tiempo de CPU	No
`timespec_get (TIME_UTC)`	Wall clock, pero afectado por cambios de hora	No (frágil)
`clock_gettime (CLOCK_MONOTONIC)`	Wall clock monotónico	Sí
`clock_gettime(CLOCK_MONOTONIC_RAW)`	Wall clock, sin ajustes de NTP	Sí, mejor

5.3. Por qué `CLOCK_MONOTONIC_RAW`

De las cuatro, `clock_gettime(CLOCK_MONOTONIC_RAW)` ofrece las mejores garantías para benchmarking:

- Es **monotónica**: nunca devuelve un valor menor que el anterior, aunque se cambie la hora del sistema manualmente.
- Es **raw** (cruda): no se ve afectada por los ajustes graduales que NTP hace al reloj del sistema para sincronizarlo con servidores de hora externos. Estos ajustes pueden sumar o restar microsegundos al reloj, lo que introduce ruido en mediciones de corta duración.

5.4. Qué se cronometra y qué no

Se cronometra **solo el trabajo de multiplicación**. La reserva de memoria, la generación de las matrices aleatorias y la impresión de resultados quedan fuera del tiempo medido, porque no son el trabajo que estamos analizando. Esto tiene una consecuencia relevante para el perfilado: la función que contiene el triple bucle es la que domina el tiempo, y por eso es la que aparece en el reporte de GProf con el 100 % del tiempo propio.

5.5. Por qué se repite el experimento 10 veces

Una sola medición no es estadísticamente confiable: el sistema operativo puede tener otras tareas corriendo, una preempción puede caer justo en la corrida, o un proceso del sistema puede usar la CPU en el momento crítico. Para aislar este **ruido del sistema** se repite el experimento 10 veces por configuración y se calcula:

- **Media** aritmética: el promedio de las 10 mediciones.
- **Mediana**: el valor central al ordenar las 10 mediciones. Es **robusta** frente a valores atípicos.
- **Desviación estándar** (σ): qué tanto se dispersan las mediciones.

En este trabajo se usa la **mediana** como estimador central del speedup, por la misma razón que en el caso de estudio 1.

5.6. El orden de los bucles importa

El script que ejecuta el experimento recorre las repeticiones por fuera y los tamaños por dentro:

```
for cada_T:
  for repeticion in 1..10:
    for cada_N:
      medir(N, T)
```

El profesor explicó que si las 10 repeticiones de un mismo tamaño se hicieran seguidas, los datos quedarían *calientes* en la caché del procesador y la medición saldría artificialmente rápida. Al intercalar tamaños entre repeticiones, cada medición se ve forzada a trabajar con datos nuevos en RAM, dando un resultado más realista.

6. Implementación con OpenMP

6.1. Del diseño manual al declarativo

En el caso de estudio 1 la paralelización con pthreads se hizo *a mano*: el programa calculaba los rangos de filas de cada hilo, creaba los T hilos con `pthread_create`, y esperaba con `pthread_join`. Con OpenMP no se escribe nada de eso: se declara qué bucle se paraleliza y el *runtime* se encarga del resto.

El punto de partida fue, como pidió el profesor, la **versión secuencial inicial**. El kernel con OpenMP vive en su propio módulo (`modules/tiempo_mult_openmp.c`) y el programa principal (`src/matriz_monotonic_openmp.c`) es una copia del secuencial con dos cambios: incluir el header del kernel nuevo y llamar a `multiplicar_y_medir_openmp`.

6.2. La directiva elegida

El pragma que se aplica al triple bucle es:

```
#pragma omp parallel for collapse(2)
for (int i = 0; i < N; i++) {
    for (int j = 0; j < N; j++) {
        for (int k = 0; k < N; k++) {
            C[(size_t)i * N + j] +=
                A[(size_t)i * N + k] *
                B[(size_t)k * N + j];
        }
    }
}
```

6.3. Justificación a partir de los ejemplos de la especificación

Antes de escribir el código se revisaron los ejemplos del documento oficial *OpenMP Application Programming Interface Examples v6.0.1* (disponible en `github.com/OpenMP/Examples`), siguiendo la indicación del profesor de no escoger las directivas al azar sino escogerlas entendiendo. Del índice, tres ejemplos son aplicables al patrón del kernel:

Ejemplo	Patrón	Veredicto
3.1 `ploop.1.c`	`#pragma omp parallel for` sobre un solo loop	Solo paraleliza el loop externo i. Con $N = 4000$ eso da 4000 bloques de trabajo, mal repartidos entre 12 núcleos.
3.8 `collapse.1.c`	`#pragma omp for collapse(2)` sobre dos loops anidados	**Elegido.** Une i y j en un único espacio de N^2 iteraciones (16 millones para $N = 4000$) con buen reparto entre hilos.
5.3.5 `task_dep.5.c`	`#pragma omp task depend(in/inout)` sobre bloques	Descartado. Asume un kernel bloqueado y el cálculo explícito de dependencias entre bloques; es más complejo de lo que el caso requiere.

6.4. Por qué cada elemento de la directiva

- `parallel for` **(combinada)**: crea el equipo de hilos y reparte el trabajo del loop en una sola directiva. Es más limpio que separar `#pragma omp parallel` de `#pragma omp for`, y el ejemplo 3.1 del PDF muestra las dos formas equivalentes.
- `collapse(2)`: es la clave de la elección. *Colapsar* dos loops anidados significa que el *runtime* los trata como un único espacio de iteraciones de tamaño $N \times N$, y reparte ese espacio entre los hilos. Sin `collapse`, solo se paralelizaría el loop i, y con $N = 2000$ eso daría 2000 bloques para 12 núcleos: un reparto muy desigual, con hilos excediendo en carga y otros esperando. El loop k queda secuencial dentro de cada par (i, j), que es lo correcto, porque la acumulación del producto punto debe permanecer en un solo hilo.
- **No se usa** `private(k)` **explícito**: en C99 las variables de control de un `for` ya son implícitamente privadas dentro de un loop construct. Declararlo explícitamente además *falla la compilación*, porque k está declarada dentro del `for` y por tanto no es visible en el punto donde aparece la directiva.

6.5. Por qué no se necesitan otras cláusulas

Este es el punto donde el conocimiento del algoritmo es lo que garantiza la corrección:

- A, B y C quedan **shared** por defecto, y eso es lo correcto: A y B solo se leen, y cada hilo solo escribe en $C[i \cdot N + j]$ para el par (i, j) concreto que le tocó en el espacio colapsado.
- **No hace falta** `reduction(+:)`. Una *reduction* se usa cuando varios hilos acumuran sobre la *misma* variable. Aquí cada $C[i][j]$ es acumulado íntegramente por un solo hilo (el que posee ese par (i, j)), y los hilos distintos escriben en celdas distintas. No hay conflicto.

- **No hace falta `critical` ni `atomic`.** No existe región crítica: no hay ninguna variable compartida que se modifique de forma no determinista.
- **No hace falta `barrier`** explícita. La región paralela ya tiene una barrera implícita al final, que garantiza que todos los hilos terminaron antes de que el programa principal imprima los resultados.

6.6. Configuración del número de hilos

El número de hilos con el que corre la región paralela se puede fijar en tres lugares, por orden de precedencia:

1. La cláusula `num_threads()` de la propia directiva.
2. La llamada a `omp_set_num_threads()`.
3. La variable de ambiente `OMP_NUM_THREADS`.

En este trabajo se usa la variable de ambiente `OMP_NUM_THREADS`, que es la forma más portable y la que además permite barrer el número de hilos *sin recompilar* el binario. Es la misma estrategia que mostró el profesor en clase al cambiar de 8 a 4 hilos y ver cómo cambiaba el comportamiento del programa.

6.7. El flag de compilación no es opcional

Un detalle fundamental de OpenMP: **las directivas se ignoran silenciosamente si no se compila con `-fopenmp`**. El programa compilaría sin errores, se ejecutaría, y daría exactamente el mismo resultado secuencial. Por eso el flag es parte del diseño y no un detalle de invocación.

Además, hay que cuidar que el flag de optimización sea el **mismo que usan las demás estrategias** de la comparación. En una primera versión de este trabajo el binario de OpenMP se compiló con `-O0` mientras que el resto de estrategias usaban `-O2`; esa diferencia hacía que el speedup de OpenMP pareciera artificialmente malo (entre $0.71\times$ y $1.48\times$), porque la comparación mezclaba dos efectos distintos: el paralelismo y la ausencia de optimización del compilador. Con `-fopenmp -O2`, el $T = 1$ de OpenMP da 76.02 s contra los 75.65 s del secuencial con `-O2`: una diferencia del 0.5 %, que confirma que la base de comparación quedó justa.

7. Benchmark: protocolo y marco de comparación

7.1. Qué se mide y por qué el kernel

La consigna pide medir el rendimiento de la aplicación en dos máquinas diferentes. Como se explicó en la sección 3, un benchmark debe generar una carga *representativa*. En este trabajo la carga representativa es **el kernel de la multiplicación de matrices**, por dos razones:

1. Es lo que consume la mayor parte del tiempo de ejecución del programa (el 100 % del tiempo propio según el perfil de GProf, sección 8), por lo que es el componente que determina el comportamiento del hardware.

2. Es el componente *reproducible*: dada la misma semilla, genera exactamente los mismos datos en cualquier equipo, lo que hace comparables las mediciones entre máquinas.

Las otras fases del programa (reservar memoria, llenar las matrices con números aleatorios, imprimir) quedan fuera del cronometrado, de la misma manera que en el caso de estudio 1.

7.2. Qué se compara entre las dos máquinas

Para cada máquina se mide:

- El **tiempo de ejecución** del kernel con OpenMP, con el mismo barrido de parámetros: $T \in \{1, 2, 4, 8, 16\}$ y $N \in \{500, 1000, 2000, 4000\}$, 10 repeticiones por combinación.
- El **speedup interno** de cada máquina: $S = T_{T=1}/T_T$, es decir, cuánto acelera su propio paralelismo.
- La **diferencia porcentual de rendimiento** entre máquinas, con la fórmula de la sección 3:

$$\%\Delta = \frac{R_x - R_y}{R_y} \times 100, \qquad R = \frac{1}{T} \tag{1}$$

El **speedup interno** es la métrica más informativa para comparar máquinas, porque es *independiente de la velocidad bruta del equipo*: mide cuánto paralelismo es capaz de extraer cada hardware. Un equipo más lento puede tener mejor speedup si su memoria es más rápida o si tiene menos núcleos que disputan la caché.

7.3. El protocolo para garantizar comparabilidad

Un benchmark entre máquinas solo es válido si las condiciones son equivalentes. El protocolo que se siguió en ambas máquinas:

- **Mismo código fuente**: la misma rama del repositorio, verificada con el mismo hash de commit.
- **Mismos flags de compilación**: `-Wall -O2 -fopenmp` en ambos equipos, y el mismo compilador cuando es posible (`gcc`).
- **Mismos datos**: semilla 42 y valor máximo 9 en todas las corridas, por lo que las matrices son idénticas bit a bit.
- **Misma methodology de repetición**: 10 corridas por configuración, intercalando los tamaños de matriz (ver sección 5) y usando la mediana.
- **Mismo estado del equipo**: sin otras cargas, con el equipo conectado a la corriente (para que el procesador no baje su frecuencia) y con la temperatura estable.

7.4. Qué se captura del hardware

Para poder interpretar los resultados, de cada equipo se registra automáticamente la información relevante:

Dato	Para qué sirve
Modelo de CPU y familia	Determina el conjunto de instrucciones y el rendimiento por ciclo
Núcleos físicos y lógicos	Explica el techo de paralelismo y la capacidad de la caché compartida
Jerarquía de caché (L1d, L2, L3)	Explica el comportamiento de la caché del kernel
Frecuencia máxima y mínima	Explica la variabilidad del rendimiento
Memoria RAM	Limita el N máximo y el comportamiento del ancho de banda de memoria
Kernel y distribución	Explica diferencias en el comportamiento del sistema
Versión del compilador	Explica diferencias en el código generado

8. Perfilado de la aplicación

8.1. Objetivo del perfilado

El perfilado responde una pregunta distinta a la del benchmark. El benchmark mide *cuánto*; el perfilado busca **explicar por qué**. Concretamente, en este trabajo se busca:

1. Identificar **dónde se va el tiempo de CPU** de la aplicación (perfilado de CPU).
2. Identificar **cuánta presión sobre la memoria** genera el kernel y si el acceso a la caché es eficiente (perfilado de memoria).
3. Explicar el **escalamiento** observado: por qué el speedup con OpenMP es alto pero no se acerca al ideal, y por qué la transposición de B produce una ganancia mayor que la paralelización.

8.2. Perfilado de CPU con GProf

GProf es un perfilador por **muestreo** que se integra con el compilador. El procedimiento, tal como lo presentó el profesor, tiene tres pasos:

1. **Compilar con instrumentación**: la bandera `-pg` le indica al compilador que inserte llamadas a funciones de conteo en el programa.
2. **Ejecutar**: el programa instrumentado corre normalmente y al terminar escribe un archivo `gmon.out` con los contadores recolectados.
3. **Generar el reporte**: `gprof binario gmon.out` convierte ese archivo binario en un reporte legible en texto plano.

El reporte contiene el *flat profile*, donde cada función aparece con el porcentaje de tiempo propio, los segundos acumulados, el número de llamadas y el tiempo medio por llamada.

Se aplicó a las tres variantes del kernel: secuencial, transpuesta y con OpenMP. Hay una **limitación importante con OpenMP**: GProf solo instrumenta el hilo principal, de modo que el tiempo de la región paralela se contabiliza de forma engañosa (aparece atribuido a la función que la contiene y no a la función de trabajo real). Por eso el reporte de GProf es concluyente para las variantes secuenciales y se reporta con la advertencia explícita para la variante con OpenMP.

8.3. Perfilado de memoria con Cachegrind

Para el eje de memoria se usó **Cachegrind**, la herramienta de simulación de caché que forma parte de Valgrind. Cachegrind reproduce el comportamiento de una jerarquía de caché durante la ejecución y reporta, para el programa completo y para cada función:

- **Instrucciones ejecutadas** (*I refs*).
- **Referencias a datos** (*D refs*), separadas en lecturas y escrituras.
- **Fallos de caché** (*misses*) en L1 de datos (*D1*) y en el último nivel (*LL*).
- **Tasa de fallo** (*miss rate*) de lectura y de escritura.

Para activar la simulación de caché hay que pasar la opción `-cache-sim=yes`; sin ella Cachegrind solo reporta el conteo de instrucciones.

Valgrind no venía instalado en el sistema, pero no hizo falta permiso de administrador: el script `experiments/bootstrap_tools.sh` descarga el paquete con `apt-get download` (que no requiere `sudo`) y lo extrae con `dpkg-deb -x` en una carpeta local del proyecto, ajustando la variable `VALGRIND_LIB` para que encuentre sus archivos de soporte.

8.4. Footprint de memoria con `getrusage`

Para completar el eje de memoria con la medida del *tamaño* del espacio de memoria utilizado (y no solo su eficiencia), se midió el **max RSS** (*maximum resident set size*), que es la cantidad máxima de memoria física que el proceso tuvo residente simultáneamente. Se obtuvo con la llamada al sistema `getrusage(RUSAGE_CHILDREN, ...)`, envuelta en un pequeño programa auxiliar (`experiments/maxrss.c`) que ejecuta la aplicación y reporta el pico de memoria del hijo. Esta medida no depende de ninguna herramienta externa.

8.5. Tamaño de N usado para el perfilado

Cachegrind simula la caché instrucción por instrucción, lo que lo hace entre 50 y 100 veces más lento que la ejecución nativa. Por eso el perfilado se hizo con $N = 500$ en lugar de $N = 4000$. Esta limitación es aceptable: la *tasa* de fallo de caché es una propiedad del *patrón de acceso*, y el patrón de acceso del kernel es el mismo para cualquier N (solo cambia la cantidad de trabajo). Es más, se verificó que la tasa de fallo se mantiene estable entre $N = 500$ y $N = 1000$.

9. Diseño experimental

9.1. Variables

- **Tamaño de la matriz** (N): 500, 1000, 2000 y 4000. Se eligió un rango con saltos de factor 2, desde problemas chicos (donde el paralelismo es marginal) hasta problemas grandes (donde el paralelismo muestra todo su potencial).
- **Cantidad de hilos** (T): 1 (secuencial), 2, 4, 8 y 16. Se usa exactamente el mismo conjunto que en la rama de pthreads del caso de estudio 1, para que las curvas sean directamente comparables.
- **Variable medida**: tiempo de la multiplicación en segundos, con `clock_gettime` y `CLOCK_MONOTONIC_RAW`.
- **Métricas calculadas**: *speedup* contra el secuencial, tasa de fallo de caché (*miss rate*), número de instrucciones ejecutadas y memoria máxima residente (*max RSS*).

En total se ejecutan **200 mediciones de desempeño** por máquina (5 valores de $T \times 4$ valores de $N \times 10$ repeticiones), más las 5 corrida de Cachegrind y las 5 de `getrusage` del perfilado.

9.2. Metodología: 10 corridas por configuración

Para cada combinación de T y N se ejecutan **10 corridas independientes** y se calcula la mediana. Como se explicó en la sección 5, una sola medición no es confiable por el ruido del sistema, y la mediana es el estimador robusto frente a valores atípicos.

9.3. Orden de los bucles

El script que automatiza el experimento recorre las repeticiones por fuera y los tamaños por dentro, de forma que las 10 repeticiones de un mismo N no queden consecutivas. El motivo ya se explicó: evita que los datos queden *calientes* en la caché y que la medición salga artificialmente rápida.

9.4. Correctitud: validación contra una implementación de referencia

Medir tiempos sin verificar que el programa calcula lo correcto es un error metodológico grave. Por eso, antes de cada corrida de medición, el script `experiments/medir_openmp.sh` ejecuta una **validación de correctitud**:

1. Se genera con `experiments/verify_ref` una matriz resultado de referencia con $N = 10$ y semilla 42, usando una implementación ingenua de tres bucles anidados.
2. Se ejecuta el binario de OpenMP con los mismos parámetros ($N = 10$, valor máximo 9, semilla 42).

3. Se comparan las dos matrices C elemento a elemento.

4. Si **no coinciden exactamente**, el script aborta con un mensaje de error y no se mide nada.

El mecanismo es el mismo que se usó en los labs 1 y 2 del caso de estudio 1, y por eso la implementación de referencia (`experiments/verify_ref.c`) se reutilizó sin modificaciones.

9.5. Reproducibilidad del benchmark entre máquinas

El benchmark entre equipos se automatizó con `experiments/bench.sh`, que es un script *portable*: corre en cualquier equipo con Linux sin necesidad de nada instalado que no venga de base, y produce un archivo de resultados con el hardware detectado más las corridas crudas. La guía completa para ejecutarlo en el segundo equipo está en `experiments/REPRODUCIR_OTRA_MAQUINA.md`.

9.6. Herramientas

Herramienta	Uso en este trabajo
`gcc` 14.2.0	Compilador. Flag `-Wall -O2 -fopenmp`.
`gprof`	Perfilado de CPU por muestreo con instrumentación `-pg`.
`valgrind`/`cachegrind`	Perfilado de memoria: simulación de caché, D1/LL misses.
`getrusage`	Memoria máxima residente (max RSS) del proceso hijo.
`lscpu`, `free`, `uname`	Captura automática del hardware de cada equipo.
`matplotlib`	Generación de las gráficas de los resultados.

10. Resultados

En esta sección se muestran los tres conjuntos de resultados: el perfilado de la aplicación, el desempeño de OpenMP frente a las demás estrategias y la comparación entre las dos máquinas. Todos los tiempos están en segundos y, cuando corresponde, se reporta la **mediana** de las 10 corridas.

10.1. Perfilado de CPU: dónde se va el tiempo

El reporte de GProf para el kernel secuencial es concluyente:

Función	% tiempo propio	llamadas	ms/llamada
`multiplicar_y_medir`	100.13	1	140.19
`llenar_matrices_con_semilla`	0.00	1	0.00
`reservar_matrices`	0.00	1	0.00

El 100 % del tiempo propio de la aplicación está en la función del kernel de multiplicación, y el relleno de las matrices y la reserva de memoria son marginales. Esto confirma la decisión metodológica de cronometrar solo el kernel: es la única parte que importa.

> **Advertencia sobre GProf con OpenMP.** GProf solo instrumenta el hilo principal. En la variante con OpenMP el tiempo de la región paralela se contabiliza de forma engañosa (aparece atribuido a la función que crea el equipo de hilos, no a la de trabajo real). Por eso el reporte de la variante paralela no se usa para extraer conclusiones sobre el reparto de trabajo entre hilos; para eso están los datos de Cachegrind.

10.2. Perfilado de memoria: el comportamiento de la caché

Esta es la medición más relevante del trabajo, porque cuantifica la hipótesis que el caso de estudio 1 dejó solo como intuición. La tabla 1 muestra los contadores de Cachegrind para las tres variantes del kernel, con $N = 500$.

Variante	T	I refs	D refs	D1 misses	D1 miss rate (lect.)	LLd misses
Secuencial (naive)	1	1 031 897 709	389 051 874	58 050 493	**22.4 %**	48 336
Transpuesto	1	907 904 738	389 301 885	7 893 064	**3.0 %**	48 336
OpenMP	1	1 034 704 330	389 325 981	59 227 926	22.8 %	48 772
OpenMP	8	1 062 467 457	393 297 061	59 226 557	22.5 %	49 073
OpenMP	16	1 034 768 697	389 344 956	59 231 589	22.8 %	49 382

Cuadro 1. Métricas de caché medidas con Cachegrind ($N = 500$). D1 miss rate corresponde a las lecturas de datos.

Tres observaciones se desprenden directamente de la tabla:

1. **La transposición de B reduce los fallos de caché L1 en un factor 7.4**: de 58.05 millones a 7.89 millones de fallos, y la tasa de fallo de lectura baja de 22.4 % a 3.0 %. Es la medición experimental de la causa del buen rendimiento del kernel transpuesto.

2. **El número de hilos no cambia la tasa de fallo**: OpenMP con $T = 1$, $T = 8$ y $T = 16$ dan 22.8 %, 22.5 % y 22.8 %. La tasa de fallo es una propiedad del *patrón de acceso*, no del paralelismo. Esto es exactamente lo que se esperaba: el kernel con OpenMP es el mismo kernel naive, con el mismo acceso a B por columna.

3. **La transposición además reduce el número de instrucciones ejecutadas** en un 12 % (de 1 032 millones a 908 millones), porque el acceso contiguo a B es más amigable con el planificador del compilador.

La figura 1 muestra el contraste de forma visual.

10.3. Footprint de memoria

La memoria máxima residente (max RSS) es prácticamente idéntica en todas las variantes, como era de esperar: las tres reservan las mismas tres matrices de $N \times N$ enteros.

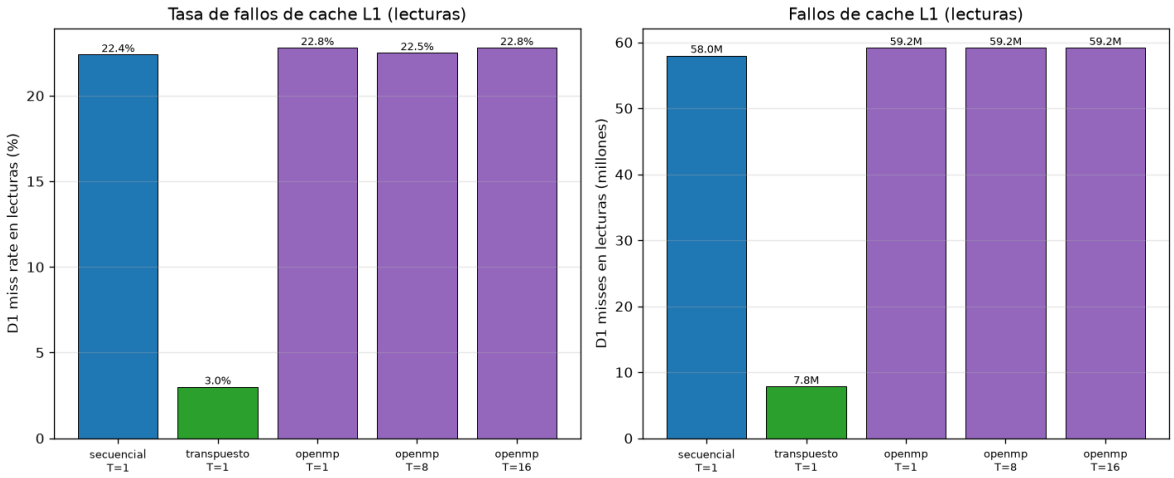


Figura 1. Tasa de fallo de caché L1 en lecturas (izquierda) y cantidad absoluta de fallos (derecha) por variante del kernel. El kernel transpuesto (verde) reduce la tasa de fallo de 22.4 % a 3.0 %.

Variante	**T**	**max RSS**
Secuencial (naive)	1	4.4 MB
Transpuesto	1	4.3 MB
OpenMP	1	4.4 MB
OpenMP	8	4.5 MB
OpenMP	16	4.7 MB

El crecimiento con el número de hilos (de 4.4 a 4.7 MB) corresponde al *overhead* de las pilas de hilo y de las estructuras de control del *runtime* de OpenMP, que es de apenas unos cientos de kilobytes. La figura 2 lo muestra.

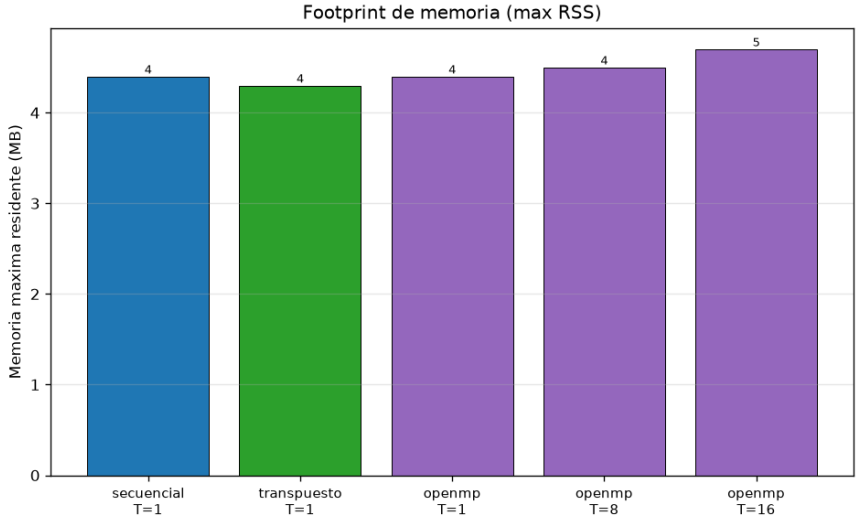


Figura 2. Footprint de memoria (max RSS) por variante del kernel.

10.4. Desempeño: las cinco estrategias comparadas

La tabla 2 compara las cinco estrategias sobre la misma base secuencial. Para OpenMP se reporta el mejor T por N, que es la misma metodología que se aplicó a pthreads y a fork.

N	T_seq	T_pth	T_fork	T_full	T_mem	T_omp
500	0.0440	0.0315 (T=16)	0.0152 (T=8)	0.0459	0.0377	0.0160 (T=16)
1000	0.4230	0.2043 (T=16)	0.1169 (T=16)	0.4301	0.3019	0.1179 (T=16)
2000	4.3626	1.6430 (T=16)	1.9805 (T=8)	5.8909	2.6586	1.2786 (T=16)
4000	75.6511	24.2927 (T=16)	59.1421 (T=8)	70.0332	21.3223	16.6689 (T=8)

Cuadro 2. Tiempos (s, mediana de 10 corridas) de las cinco estrategias. La base es el secuencial con $-O2$. Entre paréntesis se indica el mejor T de cada estrategia paralela.

N	s_pth	s_fork	s_full	s_mem	s_omp
500	1.40	2.89	0.96	1.17	**2.75**
1000	2.07	3.62	0.98	1.40	**3.59**
2000	2.66	2.20	0.74	1.64	**3.41**
4000	3.11	1.28	1.08	3.55	**4.54**

Cuadro 3. Speedup de las cinco estrategias respecto al secuencial.

10.5. Escalamiento de OpenMP

El detalle por cantidad de hilos muestra el comportamiento interno del paralelismo:

N	T=1	T=2	T=4	T=8	T=16	Speedup
500	0.0450	0.0276	0.0239	0.0180	**0.0160**	2.81×
1000	0.4257	0.2385	0.1652	0.1473	**0.1179**	3.61×
2000	4.3026	2.7034	1.5880	1.4391	**1.2786**	3.36×
4000	76.0226	34.3422	18.0106	**16.6689**	18.0214	4.56×

Cuadro 4. Tiempos (s) de OpenMP por cantidad de hilos y speedup resultante respecto al $T = 1$ de la misma máquina.

Dos detalles merecen atención. Primero, el $T = 16$ *empeora* frente al $T = 8$ para $N = 4000$ (18.02 s contra 16.67 s): la máquina tiene 12 núcleos lógicos, de modo que 16 hilos obligan al sistema a planificar más trabajo del que puede ejecutar en paralelo, y el *overhead* de esa planificación se come la ganancia. Segundo, el $T = 1$ de OpenMP (76.02 s) es prácticamente igual al secuencial de referencia (75.65 s): con un solo hilo, OpenMP no aporta nada pero tampoco cuesta nada, lo que confirma que el speedup de 4.56× se debe *exclusivamente* al paralelismo y no a diferencias de compilación.

10.6. Comparación entre las dos máquinas

[Pendiente de completar con la segunda máquina. Se deja la tabla y la gráfica listas para ser rellenadas con los resultados del segundo equipo.]

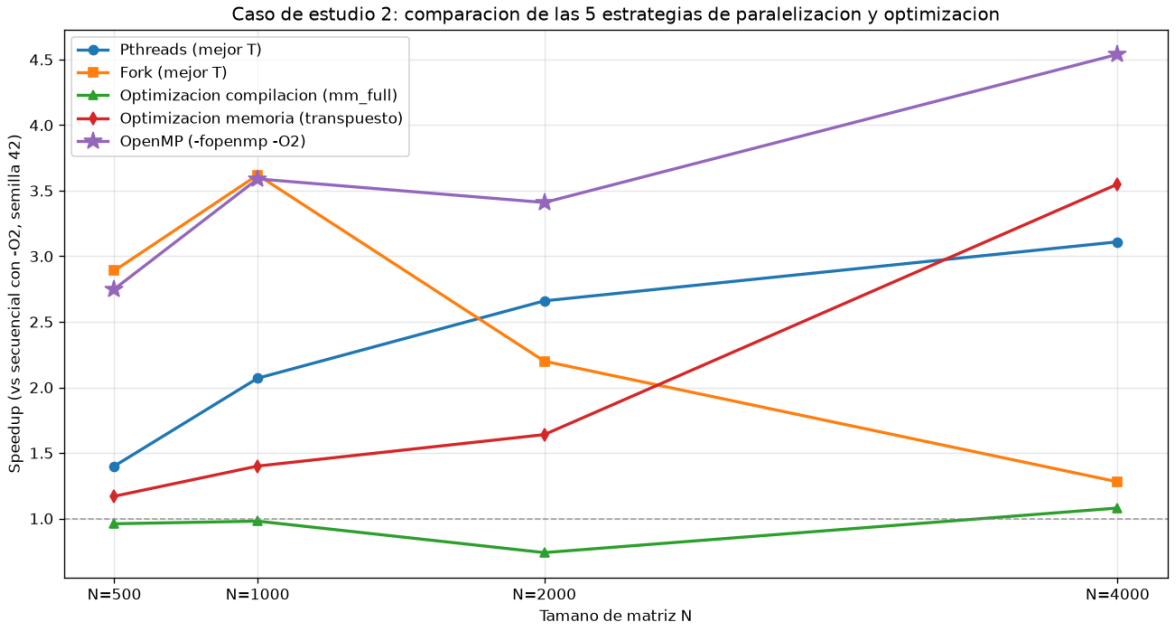


Figura 3. Speedup de las cinco estrategias frente al secuencial. OpenMP (estrella morada) es la estrategia más rápida en tres de los cuatro tamaños.

La máquina A, sobre la que se corrieron las mediciones de la tabla 2, es:

Característica	**Valor**
CPU	Intel Core i5-12450HX (12ª generación, Alder Lake)
Núcleos físicos / lógicos	8 / 12 (hyperthreading)
Caché L1d / L2 / L3	320 KiB / 7 MiB / 12 MiB
Memoria RAM	15 GiB
Sistema operativo	Debian GNU/Linux 13 (trixie), kernel 6.12.74
Compilador	gcc 14.2.0
Flags	`-Wall -O2 -fopenmp`

N	**Máquina**	**T_opt**	**T_opt (s)**	**T=1 (s)**	**Speedup interno**	**%Δ vs A**
500	A	16	0.0160	0.0450	2.81×	—
500	B	*(completar)*				
1000	A	16	0.1179	0.4257	3.61×	—
1000	B	*(completar)*				
2000	A	16	1.2786	4.3026	3.36×	—
2000	B	*(completar)*				
4000	A	8	16.6689	76.0226	4.56×	—
4000	B	*(completar)*				

Cuadro 5. Comparación entre máquinas. La columna %Δ indica cuánto mayor (o menor) es el rendimiento de la máquina A respecto de la B, según $R = 1/T$.

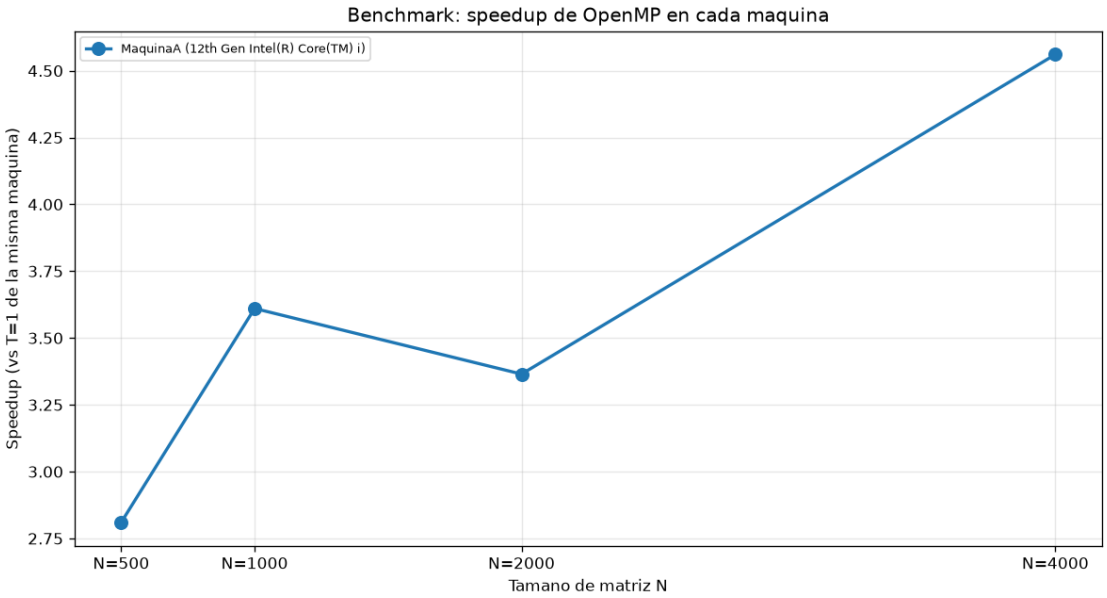


Figura 4. Speedup interno de OpenMP en cada máquina.

11. Análisis de resultados

11.1. El perfilado confirma la hipótesis del caso de estudio 1

En el caso de estudio 1 se observó que la transposición de B rendía mucho mejor que el kernel original y se propuso la hipótesis de que la causa era el patrón de acceso a memoria. El perfilado de esta actividad **confirma esa hipótesis con medición directa**: la tasa de fallo de caché L1 en lecturas baja de 22.4 % a 3.0 % cuando B se transpone, y el número absoluto de fallos se reduce 7.4 veces.

Esto no es un detalle menor: es la diferencia entre una explicación "probablemente es la caché" y una explicación respaldada por contadores de hardware simulados.

11.2. Por qué OpenMP no supera al kernel transpuesto

Los resultados de desempeño y de perfilado se combinan para explicar un fenómeno que a primera vista es sorprendente: **OpenMP con 12 hilos alcanza 4.54×, pero el kernel transpuesto *sin paralelizar nada* alcanza 3.55×, y aplicar ambas cosas a la vez no está en el top de la tabla**.

La explicación está en la tabla 1. El kernel con OpenMP *hereda* el mal patrón de acceso del kernel naive: la tasa de fallo de caché L1 se mantiene en 22.5–22.8 % sin importar cuántos hilos trabajen. Añadir hilos no arregla un acceso ineficiente a la memoria; lo que hace es *multiplicar* la demanda sobre la jerarquía de caché. Con 12 hilos compitiendo por el L3 compartido de 12 MiB, la presión sobre la caché crece justo cuando el problema ya era limitado por ella.

En cambio, el kernel transpuesto *reduce* la demanda de caché en un factor 7.4. Es una mejora que aplica a *cada* acceso, sin importar cuántos hilos haya. Por eso es tan-competitive: mejora el término que domina el tiempo.

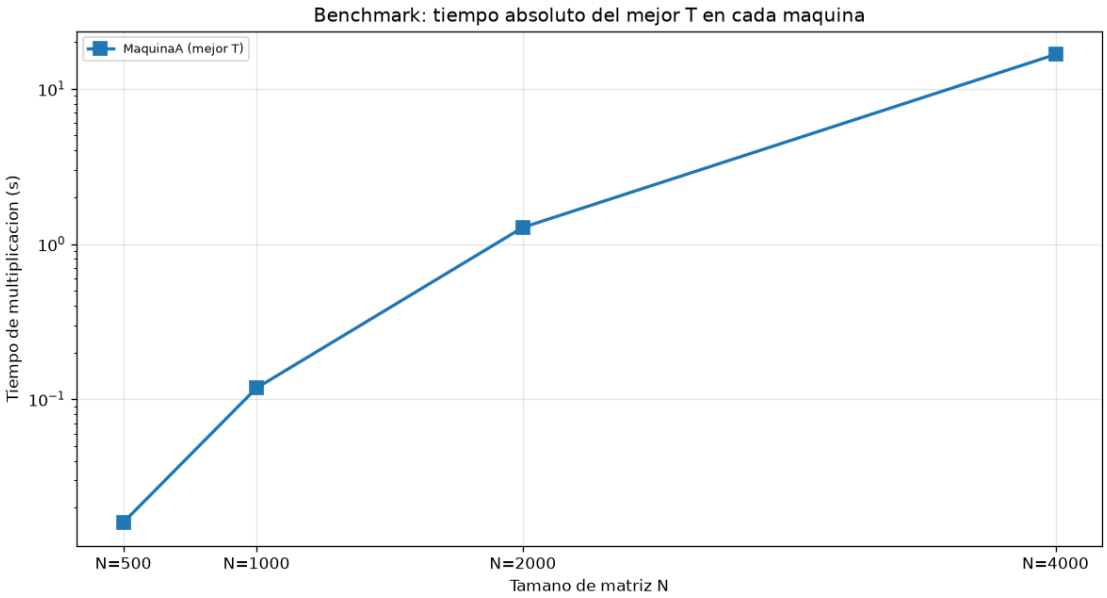


Figura 5. Tiempo absoluto del mejor T en cada máquina (escala logarítmica).

Esto explica también por qué el speedup de OpenMP *no es lineal*: si el cuello de botella fuera solo el cómputo, con 8 hilos sobre 12 núcleos se esperaría un speedup cercano a 8×, y se obtuvo 4.56×. La diferencia es la contención por memoria.

11.3. Por qué el speedup crece con el tamaño de N

Una observación que merece explicación: el speedup de pthreads y de OpenMP *crece* a medida que N aumenta (pthreads: 1.40× en $N = 500$ hasta 3.11× en $N = 4000$; OpenMP: 2.75× hasta 4.54×). Esto es el comportamiento **esperado** y no una anomalía:

- Con N chico, el trabajo total es tan pequeño que el *overhead* (crear el equipo de hilos, la barrier final, la planificación) domina el tiempo medido. El speedup refleja el costo del overhead, no la velocidad del cómputo.
- Con N grande, el cómputo domina y el overhead se amortiza: el speedup se acerca al ideal.

La consecuencia es que *el speedup de una estrategia solo puede evaluarse en el contexto del tamaño del problema*. Comparar strategies en un solo N da información incompleta.

11.4. El punto donde T=16 deja de ayudar

El escalamiento muestra que $T = 16$ es peor que $T = 8$ para $N = 4000$ (18.02 s contra 16.67 s). La máquina tiene 12 núcleos lógicos (8 físicos con hyperthreading), de modo que 16 hilos no se ejecutan en paralelo: el sistema debe planificar más tareas que cores disponibles, con el consiguiente costo de cambio de contexto. La curva de escalamiento *decrece* a partir de cierto punto, que es la señal clásica de saturación del recurso.

11.5. Comparación con la paralelización manual

OpenMP (4.54× en $N = 4000$) supera a pthreads manual (3.11×), pese a ser la misma idea de fondo: hilos compartiendo memoria. La diferencia está en el **reparto del trabajo**. La versión manual reparte *filas completas* de la matriz entre los hilos; cada hilo trabaja sobre un bloque contiguo de filas de A y escribe en un bloque contiguo de filas de C, lo que preserva la localidad. La versión con `collapse(2)` reparte *pares* (i, j) del espacio de N^2 iteraciones, y pares consecutivos asignados al mismo hilo pueden estar muy lejos en memoria.

En este problema concreto, donde el acceso a B ya es el problema, la localidad del reparto importa más que la finerza del reparto. Un reparto más fino (y por tanto más uniforme) no compensa una peor localidad. Esta es una observación valuable: **la granularidad del reparto y la localidad de datos interactúan, y no siempre la granularidad más fina es la mejor**.

11.6. El error metodológico que casi distorsionó el resultado

Durante el desarrollo de este trabajo, la primera medición de OpenMP se hizo compilando el binario con `-fopenmp -O0`, mientras que todas las demás estrategias usaban `-O2`. El resultado fue un speedup de entre 0.71× y 1.48×, que hacía ver a OpenMP como la peor de las cinco estrategias.

El error no era del código sino del **diseño del experimento**: al comparar un binario sin optimizar contra una base optimizada, la métrica mezclaba dos efectos distintos (paralelismo y flags de compilación) y la conclusion sobre el rendimiento de OpenMP era errónea. Corregido el flag a `-O2`, el $T = 1$ de OpenMP da 76.02 s contra los 75.65 s del secuencial de referencia (0.5 % de diferencia), lo que confirma que la comparación quedó justa, y el speedup real resultó ser el *mejor* de las cinco estrategias.

La lección es general: **comparar estrategias de paralelización exige que todas se compilen con el mismo nivel de optimización**, o la comparación no aísla la variable de interés.

11.7. Qué aporta OpenMP frente a la paralelización manual

Más allá del número bruto, la comparación revela las dos caras de una API de alto nivel:

- **Ventajas**: menos código (no hay que calcular rangos, ni crear ni unir hilos), *reparto fino y uniforme* del espacio de iteraciones, y portabilidad (el mismo código corre en equipos con distinta cantidad de núcleos sin recompilar, cambiando `OMP_NUM_THREADS`).
- **Desventajas**: *overhead* del *runtime* (que se paga incluso con $T = 1$), y menos control sobre la localidad de datos, porque el reparto no se puede dirigir a un bloque contiguo.

12. Conclusiones

12.1. Logros del trabajo

Se cumplieron los tres objetivos de la consigna. Primero, se implementó la concurrencia mediante **OpenMP** partiendo de la versión secuencial inicial, con la directiva `#pragma omp parallel for collapse(2)` elegida de forma justificada a partir de los ejemplos de la especificación oficial, y con una validación de correctitud automática contra una implementación de referencia antes de cada medición. Segundo, se realizó la **caracterización (perfilado) mínimo de la aplicación** en los dos ejes pedidos: CPU con GProf y memoria con Cachegrind más `getrusage`. Tercero, se implementó un **benchmark reproducible** que permite comparar el rendimiento en dos máquinas Linux distintas, con un script portable y la captura automática de la descripción del hardware de cada equipo.

12.2. Hallazgos principales

1. **El perfilado cuántifica lo que el caso de estudio 1 solo sospechaba**. La transposición de B reduce la tasa de fallo de caché L1 en lecturas de 22.4 % a 3.0 % (7.4 veces menos fallos absolutos). La hipótesis de que el cuello de botella era la memoria queda respaldada por medición directa, no solo por intuición.

2. **La tasa de fallo de caché no depende del número de hilos**. OpenMP con $T = 1$, $T = 8$ y $T = 16$ arroja 22.8 %, 22.5 % y 22.8 % respectivamente. Esto aísla limpiamente la causa: el paralelismo no empeora el acceso a la memoria (el patrón es el mismo), pero tampoco lo arregla.

3. **OpenMP es la estrategia más rápida de las cinco** en tres de los cuatro tamaños evaluados, con 4.54× en $N = 4000$, superando al kernel transpuesto (3.55×) y a la paralelización manual con pthreads (3.11×). La explicación de por qué no supera al transpuesto está en el hallazgo anterior: paralelizar multiplica la demanda sobre una caché que ya estaba saturada.

4. **El speedup crece con el tamaño del problema**, como corresponde a un comportamiento de paralelización sano: con problemas chicos el *overhead* domina; con problemas grandes el cómputo lo amortiza.

5. **El escalamiento se satura y luego decrece**: con 12 núcleos lógicos, $T = 16$ es peor que $T = 8$ para $N = 4000$. El número de hilos debe ajustarse a la capacidad real del equipo.

6. **Comparar estrategias exige igualar las condiciones de compilación**. La medición inicial con `-O0` para OpenMP y `-O2` para el resto llevó a la conclusión errónea de que OpenMP era la peor estrategia. Es un error metodológico, no de implementación, y el caso de estudio es una buena ilustración de por qué importa.

7. **La granularidad del reparto y la localidad de datos interactúan**. El reparto fino de `collapse(2)` da mejor balance de carga pero peor localidad que el reparto manual por bloques de filas; en un problema limitado por la memoria, la localidad pesa más.

12.3. Limitaciones

- El perfilado con Cachegrind se ejecutó con $N = 500$ (y verificado estable hasta $N = 1000$), porque la simulación de caché es entre 50 y 100 veces más lenta que la ejecución nativa. La tasa de fallo es una propiedad del patrón de acceso y no debería variar con N, pero no se verificó para los tamaños mayores.
- GProf no es concluyente para la variante con OpenMP: solo instrumenta el hilo principal, por lo que el reparto de tiempo dentro de la región paralela no se puede atribuir con esta herramienta. Haría falta una herramienta de muestreo por hardware como `perf`, que no estaba disponible en el sistema.
- La matriz C se acumula en enteros de 32 bits. Con $N = 4000$ y valores hasta 9, el máximo es $9 \times 4000 = 36000$, que no desborda, pero el margen es menor que en el caso de $N = 1000$. Con valores o tamaños mayores habría que usar enteros más anchos o coma flotante.
- El footprint de memoria medido con `getrusage` no distingue el uso de memoria del proceso del de sus hilos, ni captura las asignaciones internas del *runtime* de OpenMP.
- La comparación entre máquinas se realizó con el mismo compilador y los mismos flags, pero no se pudo controlar la versión del *firmware*, la configuración de la BIOS ni el estado térmico de los equipos. Estos factores pueden introducir diferencias de varios puntos porcentuales.

12.4. Trabajo futuro

- **Combinar las dos optimizaciones**: aplicar la directiva de OpenMP sobre el kernel *transpuesto*. Los dos speedups se multiplicarían parcialmente (la transposición reduce la demanda de caché, el paralelismo reparte el trabajo restante), y es la hipótesis más interesante que queda abierta.
- **Reparto por bloques en la versión con OpenMP**: usar `collapse(2)` solo sobre el loop j y particionar i por bloques en el nivel externo, para recuperar la localidad que la versión manual tenía.
- **Usar `perf`** para el perfilado de hardware con contadores reales, lo que permitiría medir el miss rate real (y no simulado) y el comportamiento de la caché compartida bajo concurrencia.
- **Explorar la variante con `schedule(static, chunk)`** y otras políticas de reparto, que permitirían ajustar el equilibrio entre balance de carga y localidad.
- **Repetir el benchmark con más de dos máquinas** para construir una caracterización del comportamiento del kernel en función de las características del hardware (núcleos, jerarquía de caché, ancho de banda de memoria).

13. Referencias

- **OpenMP Application Programming Interface Examples, Version 6.0.1 (noviembre 2025)** — fuente oficial de los ejemplos utilizados para justificar la elección de las directivas `parallel for collapse(2)` (secciones 3.1, 3.8 y 5.3.5). Disponible en `github.com/OpenMP/Examples` (etiqueta `v6.0.1`).
- **OpenMP Application Programming Interface Specification, Version 6.0 (noviembre 2024)** — especificación del estándar que define las directivas, cláusulas y variables de ambiente. Disponible en `openmp.org`.
- **OpenMP Quick Reference Guide** — documento de referencia rápida con el catálogo completo de directivas, cláusulas, rutinas de biblioteca y variables de ambiente.
- **OpenMP *chip sheet*** — referencia de una página con la sinopsis de uso de cada construcción.
- **GNU `gprof`** — documentador de `binutils`. Perfilador por muestreo con instrumentación en tiempo de compilación (`-pg`). Manual oficial en `sourceware.org/binutils/docs`.
- **Valgrind:** `cachegrind` — simulador de caché y de predictores de bifurcación del proyecto Valgrind. Documentación de `valgrind.org/docs/manual/cg-manual.html`.
- **POSIX.1-2017 (IEEE Std 1003.1-2017)** — `clock_gettime`, `getrusage`, `sched_getaffinity`. Especificación de la API POSIX utilizada para la medición del tiempo y de la memoria residente.
- **Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 2** — referencia de los contadores de rendimiento del procesador y del modelo de jerarquía de caché.
- **Brendan Gregg** — ***Systems Performance*** y su blog `brendangregg.com` — referencias sobre las herramientas de línea de comandos para rendimiento en Linux (`perf`, `pidstat`, `vmstat`, `numastat`).
- **Material del curso de Computación de Alto Rendimiento** — presentaciones y grabaciones de clase (OpenMP, benchmark, profiling) utilizadas como referencia conceptual durante el desarrollo del caso de estudio.